

◆ 1–10: Basic Array Problems (TypeScript)

1. Find the Largest Element in an Array

Approach: Scan array and track maximum.

Without inbuilt:

```
function findLargest(arr: number[]): number {
    let largest = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] > largest) largest = arr[i];
    }
    return largest;
}

console.log(findLargest([3,7,1,9,4])); // 9
```

With inbuilt:

```
function findLargestInbuilt(arr: number[]): number {
    return Math.max(...arr);
}

console.log(findLargestInbuilt([3,7,1,9,4])); // 9
```

Time: $O(n)$ | **Space:** $O(1)$

2. Find the Smallest Element in an Array

Without inbuilt:

```
function findSmallest(arr: number[]): number {
    let smallest = arr[0];
    for (let i = 1; i < arr.length; i++) {
        if (arr[i] < smallest) smallest = arr[i];
    }
    return smallest;
}

console.log(findSmallest([3,7,1,9,4])); // 1
```

With inbuilt:

```
function findSmallestInbuilt(arr: number[]): number {
    return Math.min(...arr);
}
```

```
console.log(findSmallestInbuilt([3,7,1,9,4])); // 1
```

Time: $O(n)$ | **Space:** $O(1)$

3. Find the Second Largest / Second Smallest

Without inbuilt:

```
function secondLargest(arr: number[]): number {
    let first = -Infinity, second = -Infinity;
    for (let x of arr) {
        if (x > first) {
            second = first;
            first = x;
        } else if (x > second && x !== first) {
            second = x;
        }
    }
    return second;
}

function secondSmallest(arr: number[]): number {
    let first = Infinity, second = Infinity;
    for (let x of arr) {
        if (x < first) {
            second = first;
            first = x;
        } else if (x < second && x !== first) {
            second = x;
        }
    }
    return second;
}

console.log(secondLargest([3,7,1,9,4])); // 7
console.log(secondSmallest([3,7,1,9,4])); // 3
```

With inbuilt:

```
function secondLargestInbuilt(arr: number[]): number {
    return [...new Set(arr)].sort((a,b)=>a-b)[arr.length-2];
}

function secondSmallestInbuilt(arr: number[]): number {
    return [...new Set(arr)].sort((a,b)=>a-b)[1];
}
```

4. Reverse an Array

Without inbuilt:

```
function reverseArray(arr: number[]): number[] {
    let n = arr.length;
```

```

    for (let i=0;i<n/2;i++){
        [arr[i], arr[n-i-1]] = [arr[n-i-1], arr[i]];
    }
    return arr;
}
console.log(reverseArray([1,2,3,4,5])); // [5,4,3,2,1]

```

With inbuilt:

```

function reverseArrayInbuilt(arr: number[]): number[] {
    return arr.slice().reverse();
}

```

5. Rotate Array Left/Right by k

Without inbuilt:

```

function rotateLeft(arr: number[], k: number): number[] {
    let n = arr.length;
    k %= n;
    return arr.slice(k).concat(arr.slice(0,k));
}

function rotateRight(arr: number[], k: number): number[] {
    let n = arr.length;
    k %= n;
    return arr.slice(n-k).concat(arr.slice(0,n-k));
}

```

With inbuilt (using splice):

```

function rotateArrayInbuilt(arr: number[], k: number, direction:
"left"|"right"): number[] {
    let n = arr.length;
    k %= n;
    if (direction === "left") return arr.slice(k).concat(arr.slice(0,k));
    return arr.slice(n-k).concat(arr.slice(0,n-k));
}

```

6. Sum of Elements

Without inbuilt:

```

function sumArray(arr: number[]): number {
    let total = 0;
    for (let x of arr) total += x;
    return total;
}

```

With inbuilt:

```

function sumArrayInbuilt(arr: number[]): number {
    return arr.reduce((acc,x)=>acc+x,0);
}

```

7. Average of Elements

Without inbuilt:

```
function avgArray(arr: number[]): number {
    let total = 0;
    for (let x of arr) total += x;
    return total / arr.length;
}
```

With inbuilt:

```
function avgArrayInbuilt(arr: number[]): number {
    return arr.reduce((a,b)=>a+b,0)/arr.length;
}
```

8. Count Even and Odd Numbers

Without inbuilt:

```
function countEvenOdd(arr: number[]): [number, number] {
    let even=0, odd=0;
    for (let x of arr){
        if (x%2===0) even++; else odd++;
    }
    return [even, odd];
}
```

With inbuilt:

```
function countEvenOddInbuilt(arr: number[]): [number, number] {
    const even = arr.filter(x=>x%2===0).length;
    const odd = arr.length - even;
    return [even, odd];
}
```

9. Check if Array is Sorted

Without inbuilt:

```
function isSorted(arr: number[]): string {
    let asc = true, desc = true;
    for (let i=0;i<arr.length-1;i++){
        if(arr[i]>arr[i+1]) asc=false;
        if(arr[i]<arr[i+1]) desc=false;
    }
    if (asc) return "Ascending";
    if (desc) return "Descending";
    return "Not Sorted";
}
```

With inbuilt:

```
function isSortedInbuilt(arr: number[]): boolean {  
    return JSON.stringify(arr) === JSON.stringify([...arr].sort((a,b)=>a-b)) ||  
           JSON.stringify(arr) === JSON.stringify([...arr].sort((a,b)=>b-a));  
}
```

10. Merge Two Sorted Arrays

Without inbuilt:

```
function mergeSorted(arr1: number[], arr2: number[]): number[] {  
    let i=0,j=0,res:number[]=[];  
    while(i<arr1.length && j<arr2.length){  
        if(arr1[i]<arr2[j]) res.push(arr1[i++]);  
        else res.push(arr2[j++]);  
    }  
    while(i<arr1.length) res.push(arr1[i++]);  
    while(j<arr2.length) res.push(arr2[j++]);  
    return res;  
}
```

With inbuilt:

```
function mergeSortedInbuilt(arr1:number[],arr2:number[]):number[] {  
    return [...arr1,...arr2].sort((a,b)=>a-b);  
}
```

◆ 11–20: Searching & Sorting (TypeScript)

11. Linear Search

Approach: Scan array and compare each element.

Without inbuilt:

```
function linearSearch(arr: number[], target: number): number {
    for(let i=0;i<arr.length;i++){
        if(arr[i] === target) return i;
    }
    return -1; // not found
}

console.log(linearSearch([3,7,1,9,4],9)); // 3
```

With inbuilt:

```
function linearSearchInbuilt(arr:number[], target:number): number {
    return arr.indexOf(target);
}
```

12. Binary Search (Iterative & Recursive)

Approach: Divide & conquer on sorted array.

Iterative:

```
function binarySearchIter(arr:number[], target:number): number {
    let left=0, right=arr.length-1;
    while(left<=right){
        let mid = Math.floor((left+right)/2);
        if(arr[mid]===target) return mid;
        else if(arr[mid]<target) left=mid+1;
        else right=mid-1;
    }
    return -1;
}
```

Recursive:

```
function binarySearchRec(arr:number[], target:number, left=0,
right=arr.length-1): number {
    if(left>right) return -1;
    let mid = Math.floor((left+right)/2);
    if(arr[mid]===target) return mid;
```

```

    else if(arr[mid]<target) return
    binarySearchRec(arr,target,mid+1,right);
    else return binarySearchRec(arr,target,left,mid-1);
}

```

With inbuilt: Use `arr.indexOf(target)` on sorted array (linear) or custom function above.

13. First & Last Occurrence

Without inbuilt: Binary search modification

```

function firstOccurrence(arr:number[], target:number):number{
    let left=0,right=arr.length-1, res=-1;
    while(left<=right){
        let mid=Math.floor((left+right)/2);
        if(arr[mid]===target){ res=mid; right=mid-1; }
        else if(arr[mid]<target) left=mid+1;
        else right=mid-1;
    }
    return res;
}

function lastOccurrence(arr:number[], target:number):number{
    let left=0,right=arr.length-1,res=-1;
    while(left<=right){
        let mid=Math.floor((left+right)/2);
        if(arr[mid]===target){ res=mid; left=mid+1; }
        else if(arr[mid]<target) left=mid+1;
        else right=mid-1;
    }
    return res;
}

```

14. Count Occurrences

Without inbuilt: Linear scan

```

function countOccurrences(arr:number[],target:number):number{
    let count=0;
    for(let x of arr) if(x===target) count++;
    return count;
}

```

With inbuilt:

```

function countOccurrencesInbuilt(arr:number[],target:number):number{
    return arr.filter(x=>x===target).length;
}

```

15. Find Missing Number (1...n)

Without inbuilt: Sum formula

```
function missingNumber(arr:number[],n:number):number{
    let total=n*(n+1)/2;
    let sum=arr.reduce((a,b)=>a+b,0);
    return total-sum;
}
```

With inbuilt: Same as above using `reduce`.

16. Find Duplicate Number(s)

Without inbuilt: Hash map

```
function findDuplicates(arr:number[]):number[]{
    let map: {[key:number]:boolean}={}, res:number[]=[];
    for(let x of arr){
        if(map[x]) res.push(x);
        else map[x]=true;
    }
    return res;
}
```

With inbuilt:

```
function findDuplicatesInbuilt(arr:number[]):number[]{
    let seen = new Set<number>(), res:number[]=[];
    for(let x of arr){
        if(seen.has(x)) res.push(x);
        else seen.add(x);
    }
    return res;
}
```

17. Find All Pairs Sum = k

Without inbuilt: Two nested loops

```
function pairsWithSum(arr:number[],k:number):[number,number][]{
    let res:[number,number][]=[];
    for(let i=0;i<arr.length;i++){
        for(let j=i+1;j<arr.length;j++){
            if(arr[i]+arr[j]==k) res.push([arr[i],arr[j]]);
        }
    }
    return res;
}
```

With inbuilt: Use hash set to reduce to O(n)

```
function pairsWithSumInbuilt(arr:number[],k:number):[number,number][]{
    let seen = new Set<number>(), res:[number,number][]=[];
    for(let x of arr){

```



```

        let y = k - x;
        if (seen.has(y)) res.push([y,x]);
        seen.add(x);
    }
    return res;
}

```

18. Find Triplets Sum = k

Without inbuilt: Sort + two-pointer

```

function tripletsWithSum(arr:number[],k:number):number[][]{
    arr.sort((a,b)=>a-b);
    let res:number[][]=[];
    for(let i=0;i<arr.length-2;i++){
        let left=i+1,right=arr.length-1;
        while(left<right){
            let sum=arr[i]+arr[left]+arr[right];
            if(sum===k){ res.push([arr[i],arr[left],arr[right]]); left++;
right--; }
            else if(sum<k) left++;
            else right--;
        }
    }
    return res;
}

```

19. Sort Array (Bubble / Selection / Insertion)

Bubble Sort:

```

function bubbleSort(arr:number[]):number[]{
    let n=arr.length;
    for(let i=0;i<n-1;i++){
        for(let j=0;j<n-i-1;j++){
            if(arr[j]>arr[j+1]) [arr[j],arr[j+1]]=[arr[j+1],arr[j]];
        }
    }
    return arr;
}

```

With inbuilt: arr.sort((a,b)=>a-b)

20. Sort Array (Merge / Quick)

Merge Sort:

```

function mergeSort(arr:number[]):number[]{
    if(arr.length<=1) return arr;
    let mid=Math.floor(arr.length/2);
    let left=mergeSort(arr.slice(0,mid));
    let right=mergeSort(arr.slice(mid));
}

```

```
    let res:number[]=[];
    while(left.length && right.length){
        if(left[0]<right[0]) res.push(left.shift());
        else res.push(right.shift());
    }
    return res.concat(left).concat(right);
}
```

Quick Sort:

```
function quickSort(arr:number[]):number[]{
    if(arr.length<=1) return arr;
    let pivot=arr[arr.length-1];
    let left=arr.filter(x=>x<pivot);
    let right=arr.filter(x=>x>pivot);
    let equal=arr.filter(x=>x===pivot);
    return [...quickSort(left),...equal,...quickSort(right)];
}
```

◆ 21–30: Subarray / Matrix Problems (TypeScript)

21. Maximum Subarray Sum (Kadane's Algorithm)

Approach: Track current sum and max sum.

Without inbuilt:

```
function maxSubarraySum(arr:number[]):number{
    let maxSum=arr[0], currentSum=arr[0];
    for(let i=1;i<arr.length;i++){
        currentSum=Math.max(arr[i], currentSum+arr[i]);
        maxSum=Math.max(maxSum,currentSum);
    }
    return maxSum;
}

console.log(maxSubarraySum([-2,1,-3,4,-1,2,1,-5,4])); // 6
```

With inbuilt: Use reduce to accumulate max (less common, but possible)

```
function maxSubarraySumInbuilt(arr:number[]):number{
    let maxSum=arr[0], currentSum=arr[0];
    arr.slice(1).forEach(x=>{
        currentSum=Math.max(x, currentSum+x);
        maxSum=Math.max(maxSum,currentSum);
    });
    return maxSum;
}
```

22. Subarray with Given Sum k

Approach: Sliding window (for positive numbers).

Without inbuilt:

```
function subarraySumK(arr:number[], k:number):number[][]{
    let res:number[][]=[];
    let n = arr.length;
    for(let start=0; start<n; start++){
        let sum=0;
        for(let end=start; end<n; end++){
            sum+=arr[end];
            if(sum===k) res.push(arr.slice(start,end+1));
        }
    }
    return res;
}
```

```
}
```

With inbuilt: Same, can use `slice` to extract subarrays.

23. Longest Subarray with Sum = k

Approach: Prefix sum + hashmap.

Without inbuilt:

```
function longestSubarraySumK(arr:number[], k:number):number{
    let map = new Map<number,number>();
    let sum=0, maxLen=0;
    for(let i=0;i<arr.length;i++){
        sum+=arr[i];
        if(sum===k) maxLen=i+1;
        if(!map.has(sum)) map.set(sum,i);
        if(map.has(sum-k)) maxLen=Math.max(maxLen,i-map.get(sum-k)!);
    }
    return maxLen;
}
```

24. Longest Subarray with Equal 0s and 1s

Approach: Replace 0→-1, use prefix sum + hashmap.

```
function longestEqual01(arr:number[]):number{
    let map=new Map<number,number>();
    let sum=0, maxLen=0;
    for(let i=0;i<arr.length;i++){
        let val=arr[i]===0?-1:1;
        sum+=val;
        if(sum===0) maxLen=i+1;
        if(!map.has(sum)) map.set(sum,i);
        if(map.has(sum)) maxLen=Math.max(maxLen,i-map.get(sum)!);
    }
    return maxLen;
}
```

25. Maximum Product Subarray

Approach: Track min & max product at each index.

```
function maxProductSubarray(arr:number[]):number{
    let maxProd=arr[0], minProd=arr[0], res=arr[0];
    for(let i=1;i<arr.length;i++){
        let temp=maxProd;
        maxProd=Math.max(arr[i],arr[i]*maxProd,arr[i]*minProd);
        minProd=Math.min(arr[i],arr[i]*temp,arr[i]*minProd);
        res=Math.max(res,maxProd);
    }
}
```

```
    return res;
}
```

26. Count Subarrays with Sum = k

Approach: Prefix sum + hashmap

```
function countSubarraysSumK(arr:number[], k:number):number{
    let map=new Map<number,number>();
    let sum=0, count=0;
    map.set(0,1);
    for(let x of arr){
        sum+=x;
        if(map.has(sum-k)) count+=map.get(sum-k)!;
        map.set(sum, (map.get(sum) || 0)+1);
    }
    return count;
}
```

27. Minimum Length Subarray with Sum $\geq k$

Approach: Sliding window

```
function minLengthSubarraySumK(arr:number[], k:number):number{
    let n=arr.length, left=0, sum=0, minLen=n+1;
    for(let right=0;right<n;right++){
        sum+=arr[right];
        while(sum>=k){
            minLen=Math.min(minLen, right-left+1);
            sum-=arr[left++];
        }
    }
    return minLen===n+1?0:minLen;
}
```

28. Longest Increasing Subarray

Approach: Linear scan

```
function longestIncreasingSubarray(arr:number[]):number{
    let maxLen=1, curLen=1;
    for(let i=1;i<arr.length;i++){
        if(arr[i]>arr[i-1]) curLen++;
        else curLen=1;
        maxLen=Math.max(maxLen, curLen);
    }
    return maxLen;
}
```

29. Search in Row-wise & Column-wise Sorted Matrix

Approach: Start from top-right, move left/down

```
function searchMatrix(mat:number[][], target:number):[number,number]|null{
    let row=0, col=mat[0].length-1;
    while(row<mat.length && col>=0){
        if(mat[row][col]===target) return [row,col];
        else if(mat[row][col]>target) col--;
        else row++;
    }
    return null;
}
```

30. Rotate Matrix by 90 Degrees

Approach: Transpose + reverse rows

```
function rotateMatrix(mat:number[][]):number[][]{
    let n=mat.length;
    // Transpose
    for(let i=0;i<n;i++){
        for(let j=i;j<n;j++){
            [mat[i][j], mat[j][i]] = [mat[j][i], mat[i][j]];
        }
    }
    // Reverse rows
    for(let i=0;i<n;i++) mat[i].reverse();
    return mat;
}
```

◆ 31–40: Matrix & Advanced Array Problems (TypeScript)

31. Print Matrix in Spiral Order

Approach: Maintain boundaries (top, bottom, left, right).

```
function spiralOrder(matrix:number[][]):number[]{
    let res:number[]=[];
    let top=0, bottom=matrix.length-1;
    let left=0, right=matrix[0].length-1;

    while(top<=bottom && left<=right){
        for(let i=left;i<=right;i++) res.push(matrix[top][i]);
        top++;
        for(let i=top;i<=bottom;i++) res.push(matrix[i][right]);
        right--;
        if(top<=bottom){
            for(let i=right;i>=left;i--) res.push(matrix[bottom][i]);
            bottom--;
        }
        if(left<=right){
            for(let i=bottom;i>=top;i--) res.push(matrix[i][left]);
            left++;
        }
    }
    return res;
}
```

32. Transpose of a Matrix

Approach: Swap `mat[i][j]` with `mat[j][i]`.

```
function transposeMatrix(mat:number[][]):number[][]{
    let n=mat.length, m=mat[0].length;
    let res:number[][] = Array.from({length:m}, ()=>Array(n).fill(0));
    for(let i=0;i<n;i++){
        for(let j=0;j<m;j++){
            res[j][i]=mat[i][j];
        }
    }
    return res;
}
```

33. Maximum Sum Submatrix

Approach: Reduce to 1D Kadane's using row compression

```
function maxSumSubmatrix(mat:number[][]):number{
    let rows=mat.length, cols=mat[0].length, maxSum=-Infinity;
    for(let top=0;top<rows;top++){
        let temp:number[]=Array(cols).fill(0);
        for(let bottom=top;bottom<rows;bottom++){
            for(let c=0;c<cols;c++) temp[c]+=mat[bottom][c];
            // 1D Kadane
            let sum=0;
            for(let val of temp){
                sum=Math.max(val, sum+val);
                maxSum=Math.max(maxSum, sum);
            }
        }
    }
    return maxSum;
}
```

34. Diagonal Sum of Square Matrix

Approach: Sum primary and secondary diagonals

```
function diagonalSum(mat:number[][]):number{
    let n=mat.length, sum=0;
    for(let i=0;i<n;i++){
        sum+=mat[i][i]; // primary
        if(i!==n-i-1) sum+=mat[i][n-i-1]; // secondary
    }
    return sum;
}
```

35. Set Row & Column to Zero if Element is Zero

Approach: Track zero rows/columns, then update

```
function setZeroes(mat:number[][]):number[][]{
    let rows=mat.length, cols=mat[0].length;
    let rowSet=new Set<number>(), colSet=new Set<number>();

    for(let i=0;i<rows;i++){
        for(let j=0;j<cols;j++){
            if(mat[i][j]===0){
                rowSet.add(i);
                colSet.add(j);
            }
        }
    }

    for(let i=0;i<rows;i++){
        for(let j=0;j<cols;j++){
            if(rowSet.has(i) || colSet.has(j)) mat[i][j]=0;
        }
    }
    return mat;
}
```

36. Majority Element (> n/2 times)

Approach: Boyer-Moore Voting

```
function majorityElement(arr:number[]):number{
    let count=0, candidate=0;
    for(let num of arr){
        if(count===0) candidate=num;
        count += (num===candidate)?1:-1;
    }
    // Optional verification
    let freq = arr.filter(x=>x===candidate).length;
    return freq>arr.length/2 ? candidate : -1;
}
```

37. Elements Occurring More than n/3 Times

Approach: Extended Boyer-Moore

```
function elementsMoreThanNBy3(arr:number[]):number[]{
    let candidate1:number|null=null, candidate2:number|null=null, count1=0,
    count2=0;
    for(let x of arr){
        if(candidate1===x) count1++;
        else if(candidate2===x) count2++;
        else if(count1===0){ candidate1=x; count1=1; }
        else if(count2===0){ candidate2=x; count2=1; }
        else { count1--; count2--; }
    }
    // Verify counts
    count1=arr.filter(x=>x===candidate1).length;
    count2=arr.filter(x=>x===candidate2).length;
    let res:number[]=[];
    if(count1>arr.length/3) res.push(candidate1!);
    if(count2>arr.length/3) res.push(candidate2!);
    return res;
}
```

38. Equilibrium Index (sum left = sum right)

Approach: Prefix sum

```
function equilibriumIndex(arr:number[]):number[]{
    let total=arr.reduce((a,b)=>a+b,0);
    let leftSum=0, res:number[]=[];
    for(let i=0;i<arr.length;i++){
        if(leftSum===total-leftSum-arr[i]) res.push(i);
        leftSum+=arr[i];
    }
    return res;
}
```

39. Trapping Rainwater Problem

Approach: Precompute left max and right max

```
function trapRainWater(height:number[]):number{
    let n=height.length, leftMax:number[]=Array(n).fill(0),
    rightMax:number[]=Array(n).fill(0);
    leftMax[0]=height[0]; for(let i=1;i<n;i++)
    leftMax[i]=Math.max(leftMax[i-1],height[i]);
    rightMax[n-1]=height[n-1]; for(let i=n-2;i>=0;i--)
    rightMax[i]=Math.max(rightMax[i+1],height[i]);
    let water=0;
    for(let i=0;i<n;i++) water += Math.min(leftMax[i],rightMax[i])-
    height[i];
    return water;
}
```

40. Container With Most Water

Approach: Two-pointer

```
function maxArea(height:number[]):number{
    let left=0,right=height.length-1, maxArea=0;
    while(left<right){
        let area = Math.min(height[left],height[right])*(right-left);
        maxArea = Math.max(maxArea,area);
        if(height[left]<height[right]) left++;
        else right--;
    }
    return maxArea;
}
```

Perfect! 🚀 Let's continue with **Problems 41–50** fully expanded in **TypeScript**.

◆ 41–50: Advanced Array & Interval Problems (TypeScript)

41. Maximum Difference Between Two Elements ($j > i$)

Without inbuilt:

```
function maxDifference(arr:number[]):number{
    let minVal=arr[0], maxDiff=arr[1]-arr[0];
    for(let i=1;i<arr.length;i++){
        maxDiff=Math.max(maxDiff, arr[i]-minVal);
        minVal=Math.min(minVal, arr[i]);
    }
    return maxDiff;
}
```

42. Next Greater Element

Approach: Stack

```
function nextGreaterElement(arr:number[]):number[]{
    let n=arr.length, res:number[]=Array(n).fill(-1);
    let stack:number[]=[];
    for(let i=0;i<n;i++){
        while(stack.length && arr[i]>arr[stack[stack.length-1]]){
            res[stack.pop()!]=arr[i];
        }
        stack.push(i);
    }
    return res;
}
```

43. Next Smaller Element

Approach: Stack

```
function nextSmallerElement(arr:number[]):number[]{
    let n=arr.length, res:number[]=Array(n).fill(-1);
    let stack:number[]=[];
    for(let i=0;i<n;i++){
        while(stack.length && arr[i]<arr[stack[stack.length-1]]){
            res[stack.pop()!]=arr[i];
        }
        stack.push(i);
    }
}
```

```
    }
    return res;
}
```

44. Stock Buy & Sell (Max Profit)

Approach: One-pass for max profit (buy/sell multiple)

```
function maxProfit(prices:number[]):number{
    let profit=0;
    for(let i=1;i<prices.length;i++){
        if(prices[i]>prices[i-1]) profit += prices[i]-prices[i-1];
    }
    return profit;
}
```

With cooldown / single transaction requires DP — more advanced.

45. Kth Smallest / Largest Element (QuickSelect)

```
function quickSelect(arr:number[], k:number):number{
    function partition(left:number, right:number):number{
        let pivot=arr[right], i=left;
        for(let j=left; j<right; j++){
            if(arr[j]<=pivot) [arr[i], arr[j]]=[arr[j], arr[i]], i++;
        }
        [arr[i], arr[right]]=[arr[right], arr[i]];
        return i;
    }
    function select(left:number, right:number, k:number):number{
        let pi=partition(left, right);
        if(pi===k) return arr[pi];
        else if(pi<k) return select(pi+1, right, k);
        else return select(left, pi-1, k);
    }
    return select(0, arr.length-1, k-1);
}
```

46. Merge Intervals

```
function mergeIntervals(intervals:number[][]):number[][]{
    if(intervals.length===0) return [];
    intervals.sort((a,b)=>a[0]-b[0]);
    let res:number[][]=[intervals[0]];
    for(let i=1; i<intervals.length; i++){
        if(intervals[i][0]<=res[res.length-1][1]){
            res[res.length-1][1]=Math.max(res[res.length-1][1], intervals[i][1]);
        } else res.push(intervals[i]);
    }
    return res;
}
```

47. Insert Interval

```
function insertInterval(intervals:number[][],
newInterval:number[]):number[][]{
    let res:number[][]=[], i=0;
    while(i<intervals.length && intervals[i][1]<newInterval[0])
res.push(intervals[i++]);
    while(i<intervals.length && intervals[i][0]<=newInterval[1]){
        newInterval[0]=Math.min(newInterval[0], intervals[i][0]);
        newInterval[1]=Math.max(newInterval[1], intervals[i][1]);
        i++;
    }
    res.push(newInterval);
    while(i<intervals.length) res.push(intervals[i++]);
    return res;
}
```

48. Minimum Number of Platforms (Train Arrival/Departure)

```
function minPlatforms(arrival:number[], departure:number[]):number{
    arrival.sort((a,b)=>a-b);
    departure.sort((a,b)=>a-b);
    let i=0,j=0,platNeeded=0,res=0;
    while(i<arrival.length){
        if(arrival[i]<=departure[j]) { platNeeded++; i++;
res=Math.max(res,platNeeded); }
        else { platNeeded--; j++; }
    }
    return res;
}
```

49. Count Inversions in Array

Approach: Modified Merge Sort

```
function countInversions(arr:number[]):number{
    function mergeSort(arr:number[]):[number[],number]{
        if(arr.length<=1) return [arr,0];
        let mid=Math.floor(arr.length/2);
        let [left,lCount]=mergeSort(arr.slice(0,mid));
        let [right,rCount]=mergeSort(arr.slice(mid));
        let merged:number[]=[], count=lCount+rCount, i=0,j=0;
        while(i<left.length && j<right.length){
            if(left[i]<=right[j]) merged.push(left[i++]);
            else { merged.push(right[j++]); count+=left.length-i; }
        }
        return [merged.concat(left.slice(i)).concat(right.slice(j)),
count];
    }
    return mergeSort(arr)[1];
}
```

50. Longest Consecutive Sequence

Approach: HashSet

```
function longestConsecutive(arr:number[]):number{
  let set = new Set(arr), maxLen=0;
  for(let num of arr){
    if(!set.has(num-1)){
      let curr=num, len=1;
      while(set.has(curr+1)) { curr++; len++; }
      maxLen=Math.max(maxLen,len);
    }
  }
  return maxLen;
}
```

◆ 51–60: Miscellaneous / Tricky Problems (TypeScript)

51. Rearrange Positive & Negative Numbers Alternately

Without inbuilt:

```
function rearrangePosNeg(arr:number[]):number[]{
    let pos:number[] = arr.filter(x=>x>=0);
    let neg:number[] = arr.filter(x=>x<0);
    let res:number[] = [], i=0;
    while(i<pos.length || i<neg.length){
        if(i<pos.length) res.push(pos[i]);
        if(i<neg.length) res.push(neg[i]);
        i++;
    }
    return res;
}
```

52. Rearrange Array in Wave Form

```
function waveArray(arr:number[]):number[]{
    arr.sort((a,b)=>a-b);
    for(let i=0;i<arr.length-1;i+=2){
        [arr[i],arr[i+1]]=arr[i+1],arr[i];
    }
    return arr;
}
```

53. Rearrange Array so that arr[i] = i (if possible)

```
function rearrangeIndex(arr:number[]):number[]{
    let n=arr.length;
    let res:number[]=Array(n).fill(-1);
    for(let i=0;i<n;i++){
        if(arr[i]>=0 && arr[i]<n) res[arr[i]]=arr[i];
    }
    return res;
}
```

54. Maximum Circular Subarray Sum

Approach: Max of (non-circular Kadane, totalSum - minSubarraySum)

```
function maxCircularSubarraySum(arr:number[]):number{
    let total=0, maxSum=arr[0], minSum=arr[0], currMax=arr[0],
    currMin=arr[0];
    for(let i=0;i<arr.length;i++){
        total+=arr[i];
        currMax = Math.max(arr[i], currMax+arr[i]);
        maxSum = Math.max(maxSum, currMax);
        currMin = Math.min(arr[i], currMin+arr[i]);
        minSum = Math.min(minSum, currMin);
    }
    return maxSum>0?Math.max(maxSum, total-minSum):maxSum;
}
```

55. Find Leaders in an Array

Approach: Traverse from right

```
function findLeaders(arr:number[]):number[]{
    let leaders:number[]=[], maxFromRight=-Infinity;
    for(let i=arr.length-1;i>=0;i--){
        if(arr[i]>maxFromRight){
            leaders.push(arr[i]);
            maxFromRight=arr[i];
        }
    }
    return leaders.reverse();
}
```

56. Find Peak Element

```
function findPeak(arr:number[]):number{
    let left=0,right=arr.length-1;
    while(left<right){
        let mid=Math.floor((left+right)/2);
        if(arr[mid]<arr[mid+1]) left=mid+1;
        else right=mid;
    }
    return arr[left];
}
```

57. Cyclically Rotate Array by One

```
function rotateByOne(arr:number[]):number[]{
    if(arr.length===0) return arr;
    let last=arr[arr.length-1];
    for(let i=arr.length-1;i>0;i--) arr[i]=arr[i-1];
    arr[0]=last;
    return arr;
}
```

58. Move All Zeros to End


```
function moveZeros(arr:number[]):number[]{
    let n=arr.length, j=0;
    for(let i=0;i<n;i++){
        if(arr[i]!==0){ arr[j]=arr[i]; j++; }
    }
    for(let i=j;i<n;i++) arr[i]=0;
    return arr;
}
```

59. Smallest Missing Positive Integer

```
function smallestMissingPositive(arr:number[]):number{
    let n=arr.length;
    for(let i=0;i<n;i++){
        while(arr[i]>0 && arr[i]<=n && arr[i]!==arr[arr[i]-1]){
            [arr[i],arr[arr[i]-1]]=[arr[arr[i]-1],arr[i]];
        }
    }
    for(let i=0;i<n;i++){
        if(arr[i]!==i+1) return i+1;
    }
    return n+1;
}
```

60. Implement Array Using Dynamic Memory (Simulate)

```
class DynamicArray{
    data:number[];
    size:number;
    capacity:number;
    constructor(){
        this.data=[];
        this.size=0;
        this.capacity=1;
    }
    private resize(){
        this.capacity*=2;
        let newData:number[] = new Array(this.capacity);
        for(let i=0;i<this.size;i++) newData[i]=this.data[i];
        this.data=newData;
    }
    push(val:number){
        if(this.size===this.capacity) this.resize();
        this.data[this.size]=val;
        this.size++;
    }
    get(index:number):number|null{
        if(index<0 || index>=this.size) return null;
        return this.data[index];
    }
    set(index:number,val:number){
        if(index>=0 && index<this.size) this.data[index]=val;
    }
    pop():number|null{
        if(this.size===0) return null;
        this.size--;
        return this.data[this.size];
    }
}
```

```
    }  
    length():number{  
        return this.size;  
    }  
}  
  
let arr=new DynamicArray();  
arr.push(10); arr.push(20); arr.push(30);  
console.log(arr.get(1)); // 20  
arr.pop();  
console.log(arr.length()); // 2
```
